LESSON 07 · CHEAT SHEET

Filtering, Mapping & Slicing

Java 8 → Java 21 · results use the shared course dataset
course "today" = 2026-11-07

Key rules

1. **filter** keeps elements where the predicate is true. Money: `compareTo/signum`, never `equals` (scale!). 8
2. **map**: 1 in → 1 out (count and order unchanged). **mapToInt/Long/Double** for numbers; **mapToObj/boxed** back. 8
3. **flatMap**: the function returns a *Stream*; all are merged. 1 → 0..n. 8
4. **mapMulti**: push 0..n results into a consumer; no Stream per element. 16
5. **distinct** uses `equals/hashCode`, keeps the first. Records just work. 8
6. **peek** is for debugging only: `count()` may skip it. 8
7. **limit/skip** slice by position; **takeWhile/dropWhile** by condition, on **sorted** input. 9

Operations

Operation	Shape	Note	Since
`filter(Predicate)`	n → ≤ n	stateless	8
`Predicate.not(p)`	helper	negate a method ref	11
`map(Function)`	1 → 1	type may change	8
`mapToInt/Long/Double`	1 → 1	primitive stream	8
`mapToObj · boxed`	1 → 1	back to objects	8
`flatMap(T → Stream)`	1 → 0..n	flattens	8
`mapMulti(BiConsumer)`	1 → 0..n	type witness	16
`distinct()`	n → ≤ n	stateful, equals	8
`peek(Consumer)`	1 → 1	debug only	8
`limit(n) · skip(n)`	by position	limit short-circuits	8
`takeWhile · dropWhile`	by condition	sorted input!	9

Canonical snippets

1 · Practice 3: GOLD transactions (flatMap)

```
customers.stream()
    .filter(c -> c.tier() == GOLD)
    .flatMap(c -> accountsOf(c).stream())
    .flatMap(a -> txnsOf(a).stream())
    .map(Transaction::id).toList(); // 19
```

2 · mapMulti: 0 or 1 result

```
flights.stream().<String>mapMulti((f, down) -> {
    if (f.booked() * 100 >= f.seats() * 95)
        down.accept(f.code());
}).toList();          // [SQ318, TR808, SQ21]
```

3 · Pagination (pages start at 1)

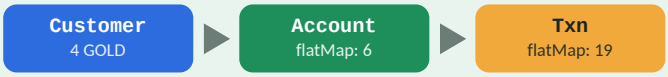
```
items.stream()
     .skip((long) (pageNo - 1) * size)
     .limit(size)
     .toList();          // past the end → []
```

takeWhile vs filter (payments, < 300.00)

```
takeWhile (source)  → [P001, P002]
filter              → 8 payments
sorted + takeWhile  → the same 8 ✓
```

P003 (310.25) stops takeWhile. takeWhile = filter **only** when the input is sorted by the same condition.

Mini diagram: map vs flatMap



`map(accountsOf)` → `Stream<List<Account>>` (10 lists). `flatMap` → `Stream<Account>`.

Real results from the course code (practice set)

Practice	Result
1 · debits > 500.00	11: T003, T005, T010 … T057
2 · names uppercase	[ALICE TAN, BALA KUMAR, …]
3 · GOLD transactions	19: T009, T012, T017, T018, T005 …
4 · distinct cities	Singapore, Kuala Lumpur, Jakarta, Colombo, Bangkok
5 · first 5 by date	T001 (08-02) … T005 (08-16)
6 · page 2, size 5	[T006 … T010]; page 13 → []

Anti-patterns → fixes

Don't	Do
`amount.equals(bd("500.00"))`	`compareTo(...) == 0`
`map(c -> accountsOf(c))`	`flatMap(c -> accountsOf(c).stream())`
`takeWhile` on unsorted input	`filter`, or sort first
`distinct` on a class without equals	a record, or equals/hashCode
save / send / count in peek	`forEach` or a loop
`mapToDouble(amount)` for money	stay with `BigDecimal`

Lesson 07 · sticky notes

Cut out, stick on your wall, and read one every morning.

Money: compareTo

500.0 equals 500.00? **false**.
compareTo → 0. Use compareTo.

map = 1 → 1

Count and order never change.
Only values (and maybe the type).

flatMap = 1 → many

Function returns a Stream?
Then flatMap, not map.

mapMulti [16]

`down.accept(x)` 0..n times.
Best for 0 or 1 result.

distinct needs equals

Records: free.
Plain class: nothing removed.

peek = debug only

`.peek(..).count()`
peeked = 0 on a List!

Paging

`skip((page-1)*size)`
`.limit(size)` · sort first

takeWhile [9]

Stops at the first failure.
Sorted input only!